Κοσσυβάκης Κωνσταντίνος – ανδριανός:4294

Φαρμάκης Μιχαήλ:2566

ΑΣΚΗΣΗ 2

Το πρόγραμμα μας εφαρμόζει τον αλγόριθμο K-Means Clustering στο ΣΔΟ σύνολο δεδομένων. Ο σκοπός του είναι να δημιουργήσει ένα σύνολο δεδομένων, να τα φορτώσει, να εκτελέσει τον αλγόριθμο K-Means για διαφορετικές τιμές του M και να αποθηκεύσει τα καλύτερα κέντρα για κάθε M και το αντίστοιχο σφάλμα SSE σε αρχεία. Τέλος έχουμε υλοποιήσει και την απεικονίσει των αρχείων με την βοήθεια της Python στον ιστότοπο Kaggle.

Για να μπορεί να γίνει το compile του προγράμματος και να φτιάξουμε τα αρχεία, θα πρέπει να υπάρχει Java Development Kit (JDK) τουλάχιστον με την έκδοση 8 και ένα IDE ή απλό terminal με πρόσβαση στις εντολές javac και java.

Τα αρχεία που υπάρχουν για την εργασία είναι το Algorithm.java που είναι ο κύριος κώδικας, το datapairsSDO.txt με τα σημεία και δημιουργείται αυτόματα από το πρόγραμμα, τα best_centers_M.txt με τα καλύτερα κέντρα για κάθε τιμή του M και δημιουργούνται αυτόματα από το πρόγραμμα, τα best_error_M.txt με την καλύτερη τιμή του σφάλματος για κάθε τιμή του M και δημιουργούνται αυτόματα από το πρόγραμμα.

Για να γίνει το compile του προγράμματος χρειάζεται η εντολή
javac Algorithm.java

και για την εκτέλεση του προγράμματος
java Algorithm

Έτσι, αφού τρέξει το πρόγραμμα,
1. Δημιουργείται το αρχείο datapairsSDO.txt με τα 1200 τυχαία σημεία.
2. Τα σημεία φορτώνονται στη λίστα pairs.
3. Εκτελείται ο K-Means για τις τιμές M = 3, 5, 7, 9, 11, 13.
4. Τα αποτελέσματα αποθηκεύονται στα best_centers_M.txt και best_error_M.txt.

Βοηθητική κλάση XYPair

Για να αναπαριστήσουμε ένα σημείο από το αρχείο χρειαζόμαστε την κλάση XYPair καθώς αντιπροσωπεύει ένα σημείο στον 2D χώρο.

```
public static class XYPair {
    public double x, y;

    public XYPair(double x, double y) {
        this.x = x;
        this.y = y;
    }
}
```

Βοηθητική κλάση RunSDO

Η κλάση RunSDO μας επιτρέπει να δημιουργούμε και να φορτώνουμε δεδομένα για τον αλγόριθμο K-Means.

Χρησιμοποιούμε ένα στατικό Random rand για να παράγουμε τις τυχαίες τιμές και έναν πίνακα pairsArray για την προσωρινή αποθήκευση των σημείων.

```
private static final Random rand = new Random();
private static final double[][] pairsArray = new double[1200][2];
```

Ο πίνακας pairsArray κρατά προσωρινά όλα τα σημεία πριν τα γράψουμε σε αρχείο. Ο αριθμός 1200 προκύπτει από το άθροισμα όλων των σημείων που παράγουμε στις διαφορετικές περιοχές.

<u>Με την μέθοδο createFile</u> δημιουργούμε το σύνολο δεδομένων με τα τυχαία σημεία.

```
int index = 0;
index = runData(index, 150, 0.75, 1.25, 0.75, 1.25);
index = runData(index, 150, 0.0, 0.5, 0.0, 0.5);
```

Κάθε κλήση της runData() μας επιτρέπει να δημιουργούμε num σημεία [x1, x2] x [y1, y2]. Κρατάμε τους index για να ξέρουμε που θα αποθηκεύσουμε τα επόμενα σημεία στον πίνακα pairsArray.

<u>Με την μέθοδο runData</u>, δημιουργούμε τα τυχαία σημεία και τα αποθηκεύουμε στον πίνακα pairsArray.

```
for (int i = 0; i < num; i++) {
    double x = x1 + rand.nextDouble() * (x2 - x1);
    double y = y1 + rand.nextDouble() * (y2 - y1);

    pairsArray[index][0] = x;
    pairsArray[index][1] = y;
    index++;
}
return index;
```

Το rand.nextDouble() μας δίνει έναν τυχαίο αριθμό μεταξύ 0 και 1.
Αυξάνουμε το index για να βάλουμε το επόμενο σημείο στην επόμενη θέση.

Με την μέθοδο loadFile, διαβάζουμε το dataset από το αρχείο και το φορτώνουμε στη λίστα pairs για τον K-Means.

```
private static void loadfile(String filename) {
    try {
        List<String> rows = Files.readAllLines(Paths.get(filename));

        for (String row : rows) {
            String[] tokens = row.trim().split("\\s+");
            if (tokens.length == 2) {
                double x = Double.parseDouble(tokens[0]);
                double y = Double.parseDouble(tokens[1]);

                pairs.add(new XYPair(x, y));
            }
        }
        System.out.println("Loaded " + pairs.size() + " pairs.");
    } catch (IOException e) {
        e.printStackTrace();
    }
```

Με την εντολή Files.readAllLines, φορτώνουμε όλες τις γραμμές του αρχείου και τις αποθηκεύουμε στο αντικείμενο rows, το οποίο είναι μία λίστα από strings.

Κύρια κλάση Algorithm

Αφού φτιάξουμε το σύνολο δεδομένων και το φορτώσουμε στην λίστα pairs, η οποία είναι static καθώς δεν αλλάζουν τα σημεία, καλούμε την μέθοδο algorithmKmeans.

Ως όρισμα στην μέθοδο algorithmKmeans δίνουμε το M = 3,5,7,9,11,13 και το runs που από την εκφώνηση είναι 50 καθώς τόσες φορές θέλουμε να τρέξει ο αλγόριθμος.

Ορίσαμε, το bestError με πολύ μεγάλη τιμή ώστε οποιοδήποτε πραγματικό SSE να είναι μικρότερο και το bestCenters για τα τελικά καλύτερα κέντρα.

```
double bestError = Double.MAX_VALUE;
double[][] bestCenters = new double[M][2];
```

Η μέθοδος initializeCenters δημιουργεί τα αρχικά κέντρα του K-Means επιλέγοντας M διαφορετικά τυχαία σημεία από τη λίστα pairs.

```
private static double[][] initializeCenters(int M) {
    double[][] centers = new double[M][2];
    Set<Integer> picks = new HashSet<>();
    for (int i = 0; i < M; i++) {
        int idx;
        do {
            idx = r.nextInt(pairs.size());
        } while (picks.contains(idx));

        picks.add(idx);
        centers[i][0] = pairs.get(idx).x;
        centers[i][1] = pairs.get(idx).y;
    }
    return centers;
}
```

Χρησιμοποιούμε το Set<Integer> για να διασφαλίσουμε ότι δεν θα επιλεγεί το ίδιο σημείο δύο φορές. Στη συνέχεια, κάνουμε μια τυχαία αρχικοποίηση των κέντρων και αποθηκεύουμε τις συντεταγμένες τους και τα επιστρέφουμε.

Το loop της initializeCenters σταματάει όταν επιλεγούν M μοναδικά κέντρα.

Ο εξωτερικός βρόχος εκτελείται πάντα M φορές, ενώ ο εσωτερικός do-while σταματάει όταν εντοπίσει ένα νέο index που δεν έχει ξανά επιλεγεί.

Στην συνέχεια, καλούμε την μέθοδο kMEANS, που υλοποιεί τον αλγόριθμο k-means. Η μεταβλητή M αντιπροσωπεύει το πλήθος των clusters, για παράδειγμα, εάν η μέθοδος καλείται με M=3, τότε έχουμε 3 ομάδες.

Η μεταβλητή N αντιστοιχεί στο πλήθος των σημείων που θα ομαδοποιήσουμε. Αφού το σύνολο δεδομένων αποτελείται από 1200 σημεία, άρα το N = 1200.

Η χρήση του assignedCluster και της αρχικοποίησης του με το −1 αποτελεί κρίσιμο κομμάτι του K-Means, γιατί δηλώνει ότι κανένα σημείο δεν ανήκει σε cluster στην αρχή, επιτρέπει τον έλεγχο της σύγκλισης και διασφαλίζει ότι ο αλγόριθμος εκτελείται σωστά από την πρώτη κιόλας επανάληψη.

Χρησιμοποιούμε την do-while για να εξασφαλίσουμε την εκτέλεση τουλάχιστον μίας πλήρους επανάληψης. Το changed γίνεται true όταν κάποιο σημείο αλλάζει ανάθεση σε σχέση με την προηγούμενη επανάληψη.
Όταν σε μια επανάληψη κανένα σημείο δεν αλλάξει, τότε σταματάμε.

Για κάθε σημείο p, υπολογίζουμε τις αποστάσεις από όλα τα M κέντρα με την distanceSquared για να αποφασίσουμε σε ποιο cluster θα ανήκει το συγκεκριμένο σημείο. Ξεκινάμε υποθέτοντας ότι το πλησιέστερο κέντρο είναι το 0 και υπολογίζουμε την απόσταση του σημείου από το πρώτο κέντρο.
Το bestDist έχει αυτή την απόσταση και ψάχνουμε αν υπάρχει πιο κοντινό κέντρο.

```
for (int i = 0; i < N; i++) {
    XYPair p = pairs.get(i);
    int bestCluster = 0;
    double bestDist = distanceSquared(p, centers[0]);
```

Πρέπει να συγκρίνουμε το σημείο με όλα τα M κέντρα, γιατί ο K-Means απαιτεί να βρούμε το κέντρο με την ελάχιστη απόσταση.
Άρα εκτελούμε M υπολογισμούς για κάθε σημείο γιατί κάθε σημείο πρέπει να ελεγχθεί απέναντι σε όλα τα clusters.

Σε κάθε επανάληψη του εσωτερικού loop, υπολογίζουμε την απόσταση του σημείου από το k-οστό κέντρο. Αν η νέα απόσταση είναι μικρότερη από την καλύτερη ως τώρα, ενημερώνουμε το bestDist και ορίζουμε το bestCluster = k.
Έτσι, στο τέλος του βρόχου θα έχουμε βρει ποιο είναι το πιο κοντινό κέντρο.

```
for (int k = 1; k < M; k++) {
    double d = distanceSquared(p, centers[k]);
    if (d < bestDist) {
        bestDist = d;
        bestCluster = k;
    }
}
```

Το assignedCluster[i], κρατάει σε ποιο cluster ανήκε το σημείο στην προηγούμενη επανάληψη.

Η **σύγκλιση** του αλγορίθμου ελέγχεται μέσω της μεταβλητής changed.
Αρχικά ορίζεται σε false και γίνεται true μόνο όταν ένα σημείο αλλάξει cluster.

```
if (assignedCluster[i] != bestCluster)
    assignedCluster[i] = bestCluster;
    changed = true;
```

Αυτό σημαίνει ότι, αν σε μια πλήρη επανάληψη κανένα σημείο δεν αλλάξει τον cluster στο οποίο ανήκει, τότε ο αλγόριθμος έχει σταθεροποιηθεί.
Άρα η σύγκλιση επιτυγχάνεται όταν το changed παραμείνει false, δηλαδή όταν οι αναθέσεις των σημείων δεν αλλάζουν πλέον και τα κέντρα δεν μετακινούνται άλλο.
Σε αυτό το σημείο ο K-Means έχει φτάσει σε ένα τοπικό ελάχιστο του SSE και τερματίζει.

Κρατάμε έναν πίνακα sum[M][2] όπου το sum[k][0] είναι το άθροισμα όλων των x σημείων που ανήκουν στο cluster k και το sum[k][1] είναι το άθροισμα όλων των y σημείων του cluster k.
Ο πίνακας count[k] αποθηκεύει πόσα σημεία ανήκουν σε κάθε cluster.

```
// Compute new centers
double[][] sum = new double[M][2];
int[] count = new int[M];

for (int i = 0; i < N; i++) {
    int k = assignedCluster[i];
    XYPair p = pairs.get(i);
    sum[k][0] += p.x;
    sum[k][1] += p.y;
    count[k]++;
}
```

<u>Αντιμετώπιση κενών clusters</u>
Κάποιες φορές, όλα τα σημεία μετακινούνται σε άλλα clusters και ένα cluster μένει τελείως άδειο. Αυτό είναι πρόβλημα, γιατί δεν μπορούμε να υπολογίσουμε τον μέσο όρο με 0 σημεία.

```
// If empty cluster, choose a random point as new center
for (int k = 0; k < M; k++) {
    if (count[k] == 0) {
        XYPair p = pairs.get(r.nextInt(N));
        centers[k][0] = p.x;
        centers[k][1] = p.y;
        count[k] = 1;
        sum[k][0] = p.x;
        sum[k][1] = p.y;
        changed = true;
    }
}
```

Αν count[k] == 0, τότε επιλέγουμε ένα τυχαίο σημείο από το σύνολο δεδομένων και το χρησιμοποιούμε ως νέο κέντρο αυτού του cluster.
Ορίζουμε ότι αυτό το cluster έχει 1 σημείο, το ίδιο το κέντρο, και θέτουμε το changed = true γιατί το μοντέλο τροποποιήθηκε.
Αυτό εξασφαλίζει ότι κανένα cluster δεν μένει κενό, κάτι που θα οδηγούσε σε σφάλματα.

<u>Υπολογισμός νέων κέντρων</u>
Γνωρίζουμε ότι το νέο κέντρο είναι ο μέσος όρος όλων των σημείων στον cluster.

```
// Update centers
for (int k = 0; k < M; k++) {
    centers[k][0] = sum[k][0] / count[k];
    centers[k][1] = sum[k][1] / count[k];
}
```

Σε αυτό το σημείο μπορούμε να υπολογίσουμε και το SSE, δηλαδή το πόσο μακριά βρίσκονται τα σημεία από τα κέντρα τους.

```
// Calculate SSE
double sse = 0;
for (int i = 0; i < N; i++) {
    XYPair p = pairs.get(i);
    int k = assignedCluster[i];

    double dx = p.x - centers[k][0];
    double dy = p.y - centers[k][1];
    sse += dx*dx + dy*dy;
}
return sse;
```

Για κάθε σημείο, βρίσκουμε το κέντρο που του αντιστοιχεί και υπολογίζουμε τις διαφορές στις συντεταγμένες, την τετραγωνική απόσταση και την προσθέτουμε στο συνολικό SSE.

Τελικά επιστρέφουμε το συνολικό SSE, το οποίο χρησιμοποιείται για να βρούμε το καλύτερο αποτέλεσμα από τις 50 επαναλήψεις και συνολικά τα καλύτερα κέντρα για κάθε M.

Οι αποστάσεις υπολογίζονται με την βοηθητική μέθοδο distanceSquared, όπου με αυτή μπορούμε να υπολογίσουμε την Ευκλείδεια απόσταση ανάμεσα σε ένα σημείο $p(x,y)$ και ένα κέντρο $c(cx,cy)$ και να αποφασίσουμε σε ποιο cluster ανήκει το κάθε σημείο.

```
private static double distance(XYPair p, double[] c) {
    double deltaX = p.x - c[0];
    double deltaY = p.y - c[1];

    return Math.sqrt((deltaX * deltaX) + (deltaY * deltaY));
}
```

Τετραγωνίζουμε τις διαφορές στους άξονες X και Y και αθροίζουμε τα τετράγωνα. Η συνάρτηση επιστρέφει έναν τύπο double που αντιπροσωπεύει την απόσταση του σημείου p από το κέντρο c.

Ο K-Means επηρεάζεται από την τυχαία αρχικοποίηση των κέντρων, οπότε με τις πολλαπλές εκτελέσεις μειώνεται η πιθανότητα να μείνουμε σε τοπικό ελάχιστο του SSE και μας επιτρέπουν να βρούμε το καλύτερο αποτέλεσμα.

Για κάθε τιμή M, αποθηκεύουμε τα κέντρα που αντιστοιχούν στο ελάχιστο SSE, ώστε μετά να μπορούμε να αναλύσουμε και να απεικονίσει τα αποτελέσματα.

Αποτελέσματα εφαρμογής του κώδικα μας

Loaded 1200 pairs.

K-means algorithm => M = 3

Run 1 => SSE = 339,080414

Run 2 => SSE = 339,080414

Run 3 => SSE = 345,149980

Run 4 => SSE = 345,149980

Run 5 => SSE = 345,149980

Run 6 => SSE = 339,080414

Run 7 => SSE = 339,080414

Run 8 => SSE = 345,149980

Run 9 => SSE = 405,989158

Run 10 => SSE = 339,080414

Run 11 => SSE = 345,149980

Run 12 => SSE = 339,080414

Run 13 => SSE = 345,149980

Run 14 => SSE = 345,149980

Run 15 => SSE = 339,080414

Run 16 => SSE = 345,149980

Run 17 => SSE = 345,149980

Run 18 => SSE = 339,080414

Run 19 => SSE = 345,149980

Run 20 => SSE = 345,149980

Run 21 => SSE = 339,080414

Run 22 => SSE = 339,080414

Run 23 => SSE = 339,080414

Run 24 => SSE = 345,149980

Run 25 => SSE = 345,149980

Run 26 => SSE = 339,080414

Run 27 => SSE = 339,080414

Run 28 => SSE = 339,080414

Run 29 => SSE = 339,080414

Run 30 => SSE = 339,080414

Run 31 => SSE = 345,149980

Run 32 => SSE = 339,080414

Run 33 => SSE = 345,149980

Run 34 => SSE = 339,080414

Run 35 => SSE = 339,080414

Run 36 => SSE = 345,149980

Run 37 => SSE = 345,149980

Run 38 => SSE = 345,149980

Run 39 => SSE = 345,149980

Run 40 => SSE = 339,080414

Run 41 => SSE = 345,149980

Run 42 => SSE = 339,080414

Run 43 => SSE = 339,080414

Run 44 => SSE = 345,149980

Run 45 => SSE = 345,149980

Run 46 => SSE = 339,080414

Run 47 => SSE = 339,080414

Run 48 => SSE = 339,080414

Run 49 => SSE = 345,149980

Run 50 => SSE = 339,080414

Best SSE => (M = 3) : 339,080414

K-means algorithm => M = 5

Run 1 => SSE = 103,328724

Run 2 => SSE = 103,328724

Run 3 => SSE = 103,328724

Run 4 => SSE = 103,328724

Run 5 => SSE = 103,328724

Run 6 => SSE = 103,328724

Run 7 => SSE = 103,328724

Run 8 => SSE = 103,328724

Run 9 => SSE = 103,328724

Run 10 => SSE = 103,328724

Run 11 => SSE = 103,328724

Run 12 => SSE = 103,328724

Run 13 => SSE = 103,328724

Run 14 => SSE = 103,328724

Run 15 => SSE = 103,328724

Run 16 => SSE = 103,328724

Run 17 => SSE = 103,328724

Run 18 => SSE = 103,328724

Run 19 => SSE = 103,328724

Run 20 => SSE = 103,328724

Run 21 => SSE = 103,328724

Run 22 => SSE = 103,328724

Run 23 => SSE = 103,328724

Run 24 => SSE = 103,328724

Run 25 => SSE = 103,328724

Run 26 => SSE = 103,328724

Run 27 => SSE = 260,711137

Run 28 => SSE = 266,565311

Run 29 => SSE = 103,328724

Run 30 => SSE = 103,328724

Run 31 => SSE = 103,328724

Run 32 => SSE = 103,328724

Run 33 => SSE = 103,328724

Run 34 => SSE = 103,328724

Run 35 => SSE = 103,328724

Run 36 => SSE = 103,328724

Run 37 => SSE = 103,328724

Run 38 => SSE = 103,328724

Run 39 => SSE = 103,328724

Run 40 => SSE = 103,328724

Run 41 => SSE = 103,328724

Run 42 => SSE = 103,328724

Run 43 => SSE = 103,328724

Run 44 => SSE = 103,328724

Run 45 => SSE = 103,328724

Run 46 => SSE = 103,328724

Run 47 => SSE = 103,328724

Run 48 => SSE = 266,565311

Run 49 => SSE = 260,707833

Run 50 => SSE = 103,328724

Best SSE => (M = 5) : 103,328724

K-means algorithm => M = 7

Run 1 => SSE = 88,277339

Run 2 => SSE = 84,515291

Run 3 => SSE = 77,683475

Run 4 => SSE = 77,683916

Run 5 => SSE = 79,167666

Run 6 => SSE = 77,683475

Run 7 => SSE = 84,347366

Run 8 => SSE = 84,314414

Run 9 => SSE = 79,160789

Run 10 => SSE = 79,160206

Run 11 => SSE = 77,682921

Run 12 => SSE = 78,945660

Run 13 => SSE = 77,683475

Run 14 => SSE = 84,376857

Run 15 => SSE = 79,152156

Run 16 => SSE = 86,120375

Run 17 => SSE = 250,916760

Run 18 => SSE = 77,674511

Run 19 => SSE = 84,368621

Run 20 => SSE = 77,683522

Run 21 => SSE = 87,033372

Run 22 => SSE = 77,725047

Run 23 => SSE = 86,009416

Run 24 => SSE = 87,460111

Run 25 => SSE = 79,144696

Run 26 => SSE = 79,154018

Run 27 => SSE = 87,460819

Run 28 => SSE = 85,011567

Run 29 => SSE = 76,590603

Run 30 => SSE = 77,870005

Run 31 => SSE = 84,368179

Run 32 => SSE = 85,105411

Run 33 => SSE = 78,362229

Run 34 => SSE = 77,685783

Run 35 => SSE = 77,899449

Run 36 => SSE = 76,644056

Run 37 => SSE = 84,376857

Run 38 => SSE = 78,955360

Run 39 => SSE = 77,713650

Run 40 => SSE = 77,697749

Run 41 => SSE = 87,677537

Run 42 => SSE = 79,144696

Run 43 => SSE = 79,177717

Run 44 => SSE = 78,943900

Run 45 => SSE = 79,160206

Run 46 => SSE = 85,679827

Run 47 => SSE = 77,873818

Run 48 => SSE = 77,713104

Run 49 => SSE = 77,682921

Run 50 => SSE = 87,033372

Best SSE => (M = 7) : 76,590603

K-means algorithm => M = 9

Run 1 => SSE = 71,353389

Run 2 => SSE = 67,879686

Run 3 => SSE = 62,758299

Run 4 => SSE = 61,720737

Run 5 => SSE = 70,441396

Run 6 => SSE = 69,187960

Run 7 => SSE = 61,219926

Run 8 => SSE = 69,671668

Run 9 => SSE = 69,127290

Run 10 => SSE = 64,625334

Run 11 => SSE = 63,308692

Run 12 => SSE = 71,531042

Run 13 => SSE = 73,320831

Run 14 => SSE = 53,542792

Run 15 => SSE = 61,735509

Run 16 => SSE = 61,121883

Run 17 => SSE = 60,308561

Run 18 => SSE = 69,542382

Run 19 => SSE = 63,297141

Run 20 => SSE = 62,712961

Run 21 => SSE = 71,748799

Run 22 => SSE = 67,742350

Run 23 => SSE = 71,886243

Run 24 => SSE = 69,141833

Run 25 => SSE = 68,714605

Run 26 => SSE = 70,327500

Run 27 => SSE = 73,281797

Run 28 => SSE = 61,223276

Run 29 => SSE = 61,074125

Run 30 => SSE = 61,068767

Run 31 => SSE = 70,862442

Run 32 => SSE = 60,934229

Run 33 => SSE = 61,017703

Run 34 => SSE = 59,891384

Run 35 => SSE = 70,624842

Run 36 => SSE = 53,531724

Run 37 => SSE = 69,527507

Run 38 => SSE = 69,528777

Run 39 => SSE = 61,083943

Run 40 => SSE = 63,307035

Run 41 => SSE = 69,409570

Run 42 => SSE = 61,102321

Run 43 => SSE = 59,819272

Run 44 => SSE = 69,088845

Run 45 => SSE = 74,023895

Run 46 => SSE = 69,236312

Run 47 => SSE = 69,614464

Run 48 => SSE = 69,269551

Run 49 => SSE = 67,998453

Run 50 => SSE = 72,146703

Best SSE => (M = 9) : 53,531724

K-means algorithm => M = 11

Run 1 => SSE = 56,082032

Run 2 => SSE = 69,141412

Run 3 => SSE = 66,092617

Run 4 => SSE = 62,373138

Run 5 => SSE = 64,902568

Run 6 => SSE = 52,655895

Run 7 => SSE = 56,584276

Run 8 => SSE = 55,827756

Run 9 => SSE = 53,393343

Run 10 => SSE = 56,446715

Run 11 => SSE = 53,864440

Run 12 => SSE = 49,192852

Run 13 => SSE = 55,012098

Run 14 => SSE = 45,492395

Run 15 => SSE = 54,157466

Run 16 => SSE = 55,083763

Run 17 => SSE = 54,784036

Run 18 => SSE = 59,006501

Run 19 => SSE = 45,266709

Run 20 => SSE = 54,481819

Run 21 => SSE = 55,504233

Run 22 => SSE = 53,614322

Run 23 => SSE = 65,763632

Run 24 => SSE = 44,851465

Run 25 => SSE = 44,585665

Run 26 => SSE = 46,836259

Run 27 => SSE = 56,063480

Run 28 => SSE = 45,173611

Run 29 => SSE = 45,123792

Run 30 => SSE = 55,832862

Run 31 => SSE = 44,566714

Run 32 => SSE = 47,068145

Run 33 => SSE = 56,280564

Run 34 => SSE = 55,638665

Run 35 => SSE = 46,105588

Run 36 => SSE = 56,303994

Run 37 => SSE = 46,207376

Run 38 => SSE = 45,108502

Run 39 => SSE = 54,591030

Run 40 => SSE = 54,046958

Run 41 => SSE = 55,693202

Run 42 => SSE = 62,662251

Run 43 => SSE = 47,465623

Run 44 => SSE = 68,853667

Run 45 => SSE = 54,809763

Run 46 => SSE = 55,609205

Run 47 => SSE = 46,856476

Run 48 => SSE = 48,695329

Run 49 => SSE = 54,938319

Run 50 => SSE = 46,754698

Best SSE => (M = 11) : 44,566714

K-means algorithm => M = 13

Run 1 => SSE = 38,596372

Run 2 => SSE = 50,481867

Run 3 => SSE = 53,104580

Run 4 => SSE = 50,046959

Run 5 => SSE = 40,931394

Run 6 => SSE = 49,803960

Run 7 => SSE = 39,881972

Run 8 => SSE = 39,758442

Run 9 => SSE = 42,425153

Run 10 => SSE = 50,749805

Run 11 => SSE = 55,717857

Run 12 => SSE = 42,491436

Run 13 => SSE = 52,274137

Run 14 => SSE = 42,650038

Run 15 => SSE = 40,879064

Run 16 => SSE = 39,843267

Run 17 => SSE = 52,232490

Run 18 => SSE = 61,140747

Run 19 => SSE = 47,616916

Run 20 => SSE = 38,592148

Run 21 => SSE = 51,561982

Run 22 => SSE = 49,901066

Run 23 => SSE = 64,924289

Run 24 => SSE = 50,193003

Run 25 => SSE = 42,973963

Run 26 => SSE = 39,111007

Run 27 => SSE = 40,028930

Run 28 => SSE = 50,794387

Run 29 => SSE = 50,223861

Run 30 => SSE = 52,126987

Run 31 => SSE = 40,575667

Run 32 => SSE = 50,396490

Run 33 => SSE = 49,696568

Run 34 => SSE = 39,455793

Run 35 => SSE = 63,119351

Run 36 => SSE = 39,818880

Run 37 => SSE = 40,633639

Run 38 => SSE = 40,812552

Run 39 => SSE = 52,950875

Run 40 => SSE = 58,598322

Run 41 => SSE = 38,566996

Run 42 => SSE = 48,944923

Run 43 => SSE = 48,750880

Run 44 => SSE = 39,928741

Run 45 => SSE = 52,504314

Run 46 => SSE = 40,161103

Run 47 => SSE = 38,596372

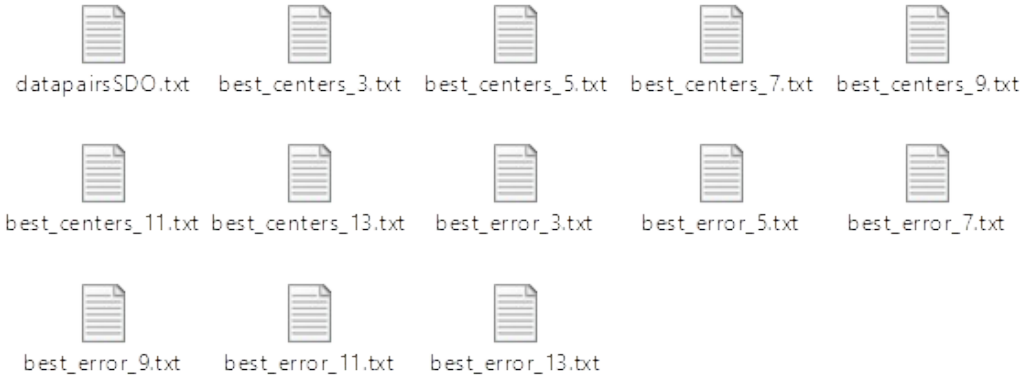
Run 48 => SSE = 50,876633

Run 49 => SSE = 49,802858

Run 50 => SSE = 51,233933

Best SSE => (M = 13) : 38,566996

Τα αρχεία που παράγονται είναι



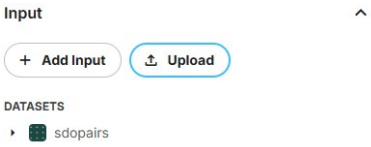
Προσθέσαμε τα αρχεία στην αναφορά μας ώστε να είναι κατανοητό το πως προέκυψαν τα διαγράμματα και τα αποτελέσματα που πέτυχε ο κώδικας μας.

Απεικόνιση σχημάτων με Python

Για την απεικόνιση των αρχείων που παράγει ο κώδικας μας χρησιμοποιήσαμε τον ιστότοπο Kaggle, οπότε και το script για την απεικόνιση των δεδομένων το κάναμε με την γλώσσα προγραμματισμού Python.

Αρκεί να φτιάξουμε ένα καινούργιο notebook, να κάνουμε επικόλληση τον κώδικα Python από την αναφορά στο notebook, να ανεβάσουμε τα αρχεία στο Kaggle και να κάνουμε run το κελί για να εμφανιστούν τα διαγράμματα.

Εδώ ανεβάζουμε τα αρχεία που προέκυψαν από το συγκεκριμένο run σαν σύνολο δεδομένων για να μπορεί ο κώδικας να τα διαβάσει και να απεικονίσει τα δεδομένα.



Για κάθε τιμή του αριθμού ομάδων M = 3,5,7,9,11,13 εμφανίσαμε σε γράφημα τα σημεία του συνόλου δεδομένων και τα κέντρα που παρήγαγε ο K-Means.
Τα δεδομένα απεικονίζονται με τον χαρακτήρα +, ενώ τα κέντρα των clusters με τον χαρακτήρα *.
Κάθε cluster επισημαίνεται με διαφορετικό χρώμα για να είναι ευδιάκριτη η ομαδοποίηση.

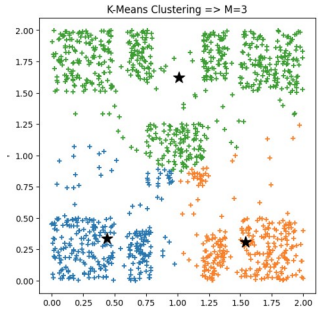
Από τα διαγράμματα παρατηρούμε ότι για τις μικρές τιμές του M πολλά διαφορετικά υποσύνολα σημείων αναγκάζονται να ενωθούν σε λίγες ομάδες.
Όσο αυξάνεται το M, τα κέντρα αρχίζουν να τοποθετούνται σε πιο «φυσικές» περιοχές του συνόλου δεδομένων.
Όταν το M είναι κοντά στο 9, που είναι και ο πραγματικός αριθμός των clusters στο σύνολο δεδομένων ΣΔΟ, τα κέντρα ευθυγραμμίζονται με τις πραγματικές συγκεντρώσεις των σημείων.

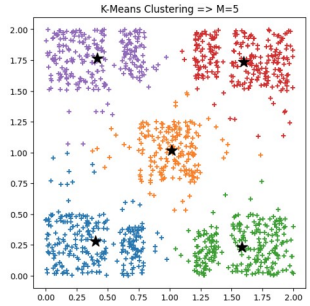
Συμπέρασμα

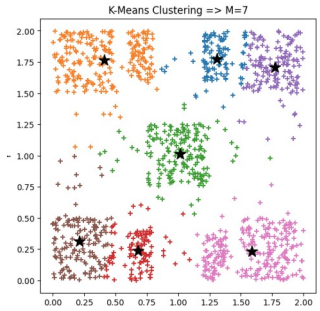
Το σφάλμα ομαδοποίησης SSE πάντα μειώνεται όσο αυξάνεται το M, επειδή τα περισσότερα clusters σημαίνουν ότι τα σημεία θα βρίσκονται πιο κοντά σε κάποιο κέντρο. Ωστόσο, για M=9 και έπειτα, το σφάλμα ομαδοποίησης SSE μειώνεται πάρα πολύ λίγο σε σχέση με τα M πριν το 9, όπου εκεί μειώνεται απότομα. Άρα, θα λέγαμε ότι υπάρχει μία σταθεροποίηση του SSE για M=9 και έπειτα, οπότε μπορούμε να πούμε ότι ο πραγματικός αριθμός ομάδων είναι 9.

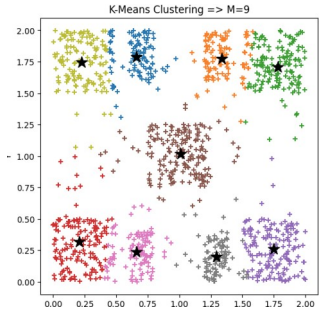
```
import numpy as np

import matplotlib.pyplot as plt


# Φόρτωση σημείων

points = np.loadtxt("/kaggle/input/sdopairs/datapairsSDO.txt")

errors = []

clusters = [3, 5, 7, 9, 11, 13]


for M in clusters:

    # Φορτώνουμε τα κέντρα

    centers = np.loadtxt(f"/kaggle/input/sdopairs/best_centers_{M}.txt")


    # Φορτώνουμε το σφάλμα

    with open(f"/kaggle/input/sdopairs/best_error_{M}.txt") as f:

        raw = f.readline().split("=")[1].strip()

        raw = raw.replace(",", ".")

        errors.append(float(raw))
```

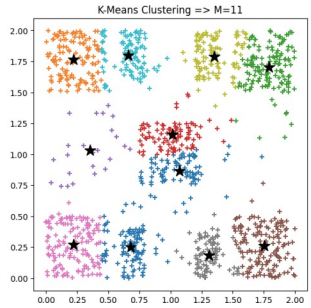
```
    # Ανάθεση σημείων σε clusters
    cluster_assignment = np.argmin(np.sum((points[:, None, :] -
centers[None, :, :])**2, axis=2), axis=1)

    # Plot
    colors = plt.cm.tab10.colors
    plt.figure(figsize=(6, 6))

    for k in range(M):
        pts = points[cluster_assignment == k]
        plt.scatter(pts[:, 0], pts[:, 1], marker="+", color=colors[k % 10])

    plt.scatter(centers[:, 0], centers[:, 1], marker="*", s=200, color="black")
    plt.title(f"K-Means Clustering => M={M}")
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.show()

# Γράφημα SSE
plt.figure(figsize=(7, 5))
plt.plot(clusters, errors, marker="o")
plt.title("Μεταβολή SSE ως προς τον αριθμό clusters")
plt.xlabel("Αριθμός Clusters M")
plt.ylabel("SSE")
plt.grid(True)
plt.show()
```

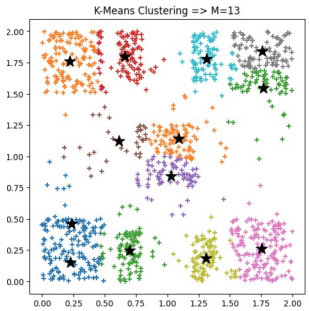
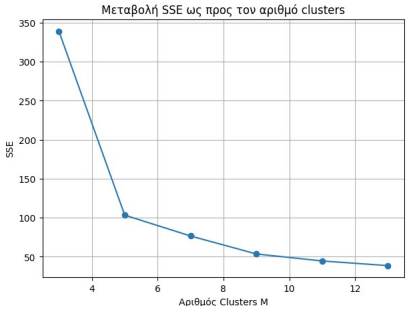

K-Means Clustering => M=3


K-Means Clustering => M=5


K-Means Clustering => M=7


K-Means Clustering => M=9


K-Means Clustering => M=11



Βάσει των αποτελεσμάτων το διάγραμμα που δείχνει πώς μεταβάλλεται το σφάλμα ομαδοποίησης με τον αριθμό των ομάδων.